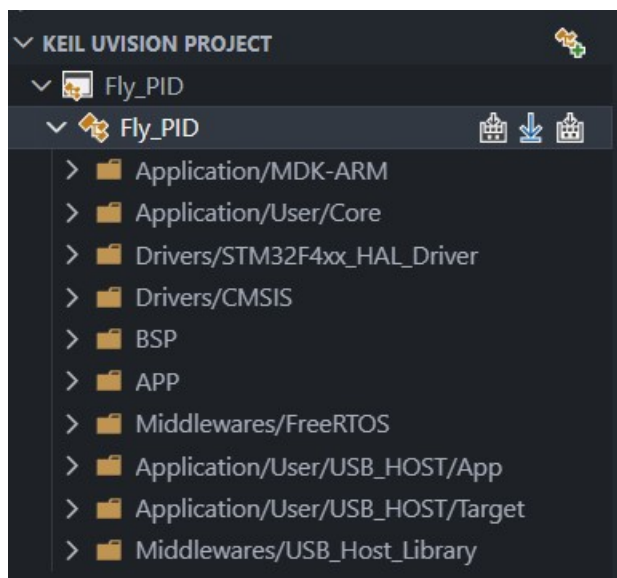


文件夹功能说明

重点文件使用红色字体标红。



- Application/MDK-ARM
由CubeMX生成，内存放的stm32启动文件，仅研究四轴应用不必理会
- Application/User/Core
由CubeMX生成，主要重点关注文件夹内的 `main.c` 和 `freertos.c` 文件。其余文件为驱动文件，用户无需修改。
 - **main.c文件**：在main函数内部初始化所有的硬件，以及启动freertos任务调度器
 - **freertos.c文件**：创建所有飞行器需要运行的任务，创建全局变量相关的队列、事件等内容，跟飞行器的运行息息相关。若用户需要新增功能，就需要此文件做修改，比如新增一个任务，新增一个队列等。
- Drivers/STM32F4xx_HAL_Driver
由CubeMX生成，是HAL库的驱动文件，用户无需修改。如需要新增功能时，不确定某些函数的使用方法，可在这些驱动文件内找到使用说明。
- Drivers/CMSIS
由CubeMX生成，芯片配置文件，由CubeMX生成。一般情况下无需用户修改
- Middlewares/FreeRTOS
由CubeMX生成，FreeRTOS中间件，用户无需修改。如果需要修改系统的配置，比如栈大小，空闲任务优先级等内容，则需要打开 `WHEELTEC.ioc` 文件在CubeMX内配置。
- **BSP**
板级支持包文件夹，针对F570这个飞控板写的驱动文件，比如LED驱动、电机驱动等等内容。如果需要新增功能，或者对某个功能的驱动修改，则是修改此文件夹下的内容
- **APP**
F570四轴飞行器的应用程序，实现F570所有功能的文件，比如平衡功能、APP交互数据功能等。当需要修改功能时，将会涉及到此文件夹内容的修改

- Application/User/USB_HOST/App

由CubeMX生成，主要用于USB手柄使用。USB应用文件，内部存放了初始化USB HOST函数。，仅研究四轴飞行器功能时，不必深究实现原理

- Application/User/USB_Host/Target

由CubeMX生成，主要用于USB手柄使用。USB配置文件，用户无需修改。内部包含了USB功能的中断等内容。

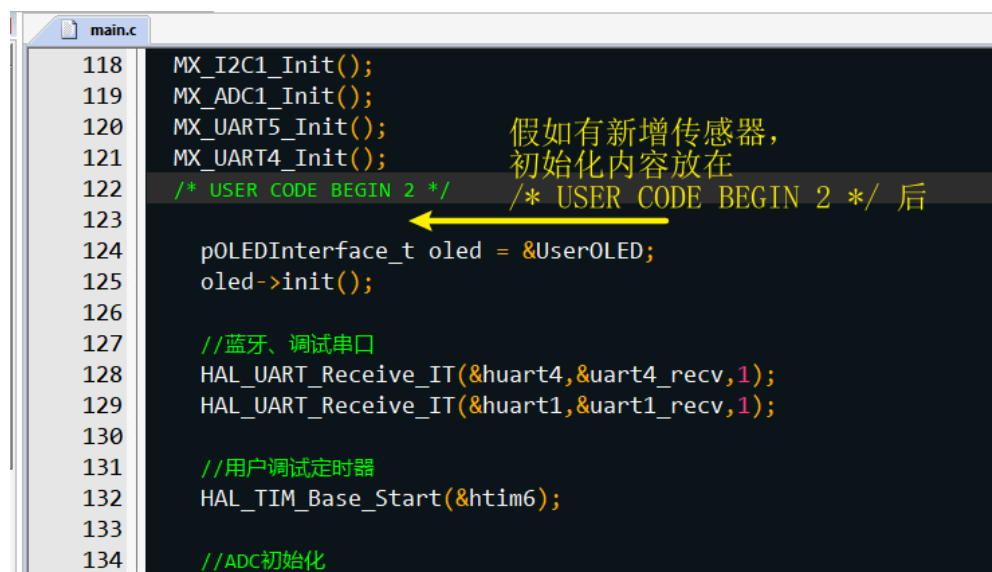
- Middlewares/USB_Host_Library

由CubeMX生成，主要用于USB手柄使用。此文件夹是USB HOST库，实现USB主机功能的必要库。仅研究四轴飞行器时，无需了解内部具体内容。

重点文件说明

1.Application/User/Core文件夹下的main.c:

如果新增了传感器，一般需要进行初始化，初始化的相关内容放置于 `/* USER CODE BEGIN 2 */` 和 `/* USER CODE END 2 */` 之间，如源代码所示：



```
118  MX_I2C1_Init();
119  MX_ADC1_Init();
120  MX_UART5_Init();
121  MX_UART4_Init();
122  /* USER CODE BEGIN 2 */
123
124  pOLEDInterface_t oled = &UserOLED;
125  oled->init();
126
127  //蓝牙、调试串口
128  HAL_UART_Receive_IT(&huart4,&uart4_recv,1);
129  HAL_UART_Receive_IT(&huart1,&uart1_recv,1);
130
131  //用户调试定时器
132  HAL_TIM_Base_Start(&htim6);
133
134  //ADC初始化
```

2.Application/User/Core文件夹下的freertos.c:

重点关注 `MX_FREERTOS_Init` 函数，此函数是配置队列、信号量、任务等重点函数。用户可以根据函数内部的注释，比如 `/* USER CODE BEGIN RTOS_MUTEX */` 下面的内容是存放互斥量的初始化，`/* USER CODE BEGIN RTOS_QUEUES */` 是存放队列的初始化内容，`/* USER CODE BEGIN RTOS_THREADS */` 是存放任务创建的初始化内容，`/* USER CODE BEGIN RTOS_EVENTS */` 是存放事件组相关的初始化。

如果需要新增任务增加功能，就将任务创建放置于 `/* USER CODE BEGIN RTOS_THREADS */` 下面，如下图所示：

```
freertos.c
250  /* creation of defaultTask */
251  defaultTaskHandle = osThreadNew(StartDefaultTask, NULL, &defaultTask_attributes);
252
253  /* USER CODE BEGIN RTOS_THREADS */ ← 创建新的任务放置于此
254  /* add threads, ... */
255  xTaskCreate(ShowTask, "ShowTask", 128*2, NULL, osPriorityBelowNormal7, NULL);
256  xTaskCreate(SensorHandleTask, "SensorTask", 128*2, NULL, osPriorityNormal, &sensorhandleT
257  xTaskCreate(balance_task, "BalanceTask", 128*4, NULL, osPriorityNormal, &balanceTask_Hand
258  xTaskCreate(BluetoothAPPControl_task, "BTAPPTask", 128*2, NULL, osPriorityNormal, NULL);
259
260  //调试任务
261  #if ( 1 == userconfig_OPEN_CPU_USAGE_CHECK ) || ( 1 == userconfig_OPEN_STACK_CHECK
262      static uint16_t delaytime = 5000; //打印时间间隔, 单位tick
263      xTaskCreate(CpuUsageCheckTask, "DebugTask", 128, &delaytime, osPriorityAboveNormal, N
264  #endif
265  /* USER CODE END RTOS_THREADS */
266
267  /* USER CODE BEGIN RTOS_EVENTS */
268  /* add events, ... */
269  g_xEventFlyAction = xEventGroupCreate(); //创建四轴动作事件组的实例
270
271  /* USER CODE END RTOS_EVENTS */
272
273  }
```

BSP文件夹

常规的驱动文件可以自行打开浏览理解，这里不作一一解释，主要对特殊功能做一些解释。

- bsp_RTOSdebug.c 驱动文件

这个驱动可以对FreeRTOS的任务进行监测，非RTOS任务也可以，比如中断环境。可以测量某个操作的用时，也可以测量某个任务执行的频率。注意测量是毫秒级别的，太快的操作无法捕获，定时器的频率的在10000Hz，高于这个频率的操作无法捕获。这个功能可以用来检测任务的频率是否达到预期，某个传感器的输出频率是否达到预期，以及检测某些操作是否太耗时而需要优化等。

捕获某个操作用时使用示例：

```
1  //假设这里是一个FreeRTOS任务内
2  void balance_task(void* param)
3  {
4      //包含“bsp_RTOSdebug.h”后，定义两个变量，用于任务频率、时间调试捕获
5      pRtosDebugInterface_t debug = &RTOSTaskDebug;
6      RtosDebugPrivateVar debugPriv = { 0 };
7
8      float useTime = 0;
9      while(1)
10     {
11         debug->TickStart(&debugPriv); //从此刻起开始计时
12         imu->Update_9axisVal(&axis_9val); //更新imu数据
13         imu->UpdateAttitude(axis_9val, &Attitudeval); //更新姿态角
14         useTime = debug->UpdateUsedTime(&debugPriv); //结束计时，此时
           useTime内存放的就是更新imu数据这个操作的耗时
15     }
16 }
```

检测任务频率的使用示例：

```

1 void balance_task(void* param)
2 {
3     //包含“bsp_RTOSdebug.h”后，定义两个变量，用于任务频率、时间调试捕获
4     pRtosDebugInterface_t debug = &RTOSTaskDebug;
5     RtosDebugPrivateVar debugPriv = { 0 };
6
7     uint16_t taskFreq = 0;
8     while(1)
9     {
10         vTaskDelay(50);
11         taskFreq = debug->UpdateFreq(&debugPriv); //统计任务频率,此时频率数据
            存放在taskFreq变量处（此例程输出20Hz）
12     }
13 }

```

APP文件夹

- uart_callback.c

该文件主要是对串口接收的回调函数作处理，比如DMA传输完成+串口空闲中断，将接收的数据写入队列，触发对应的任务去处理数据。这里有APP数据，以及STP23L激光测距的数据。DMA搬运完这些原始数据后，会通过写入队列的方式，唤醒对应的任务去解析数据。

- sensorhandle_task.c

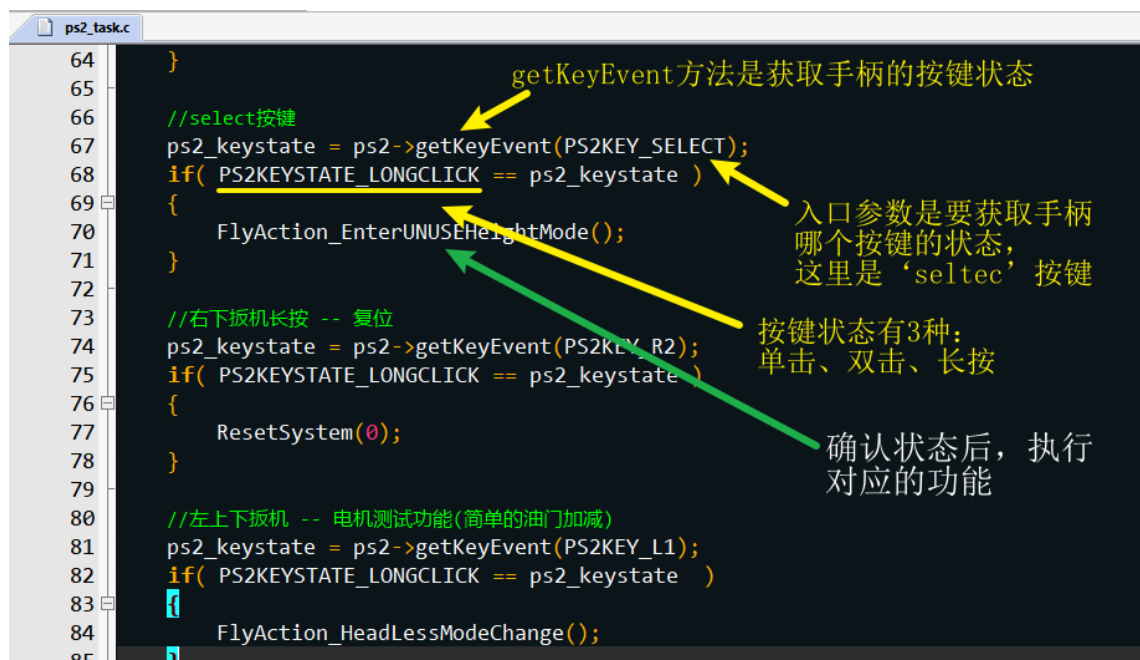
该文件是处理STP23L数据的任务。也预留了雷达数据处理的接口，但雷达还未使用。

- show_task.c

该文件是显示任务，主要是可视化一些数据，具体内容可以浏览代码，这些内容与控制无关，不影响四轴飞行器的控制。同时这个显示的任务优先级是最低的，只有CPU处理完所有控制相关的内容后，才会去执行它。

- ps2_task.c

该文件是PS2手柄操作的回调函数任务，如果用户有需要修改或新增PS2控制的操作，则在此文件夹内修改。所有按键的名称可以在 bsp_ps2.h 内查看。同时ps2按键有两个操作方法，getKeyEvent 是获取按键的事件，比如单击、双击、长按。getKeyState 则是获取按键的状态，按下是 PS2KEY_PressDOWN，未按下是 PS2KEY_PressUP。以下是对某个按键的简单示例说明：



- bluetoothControl_task.c

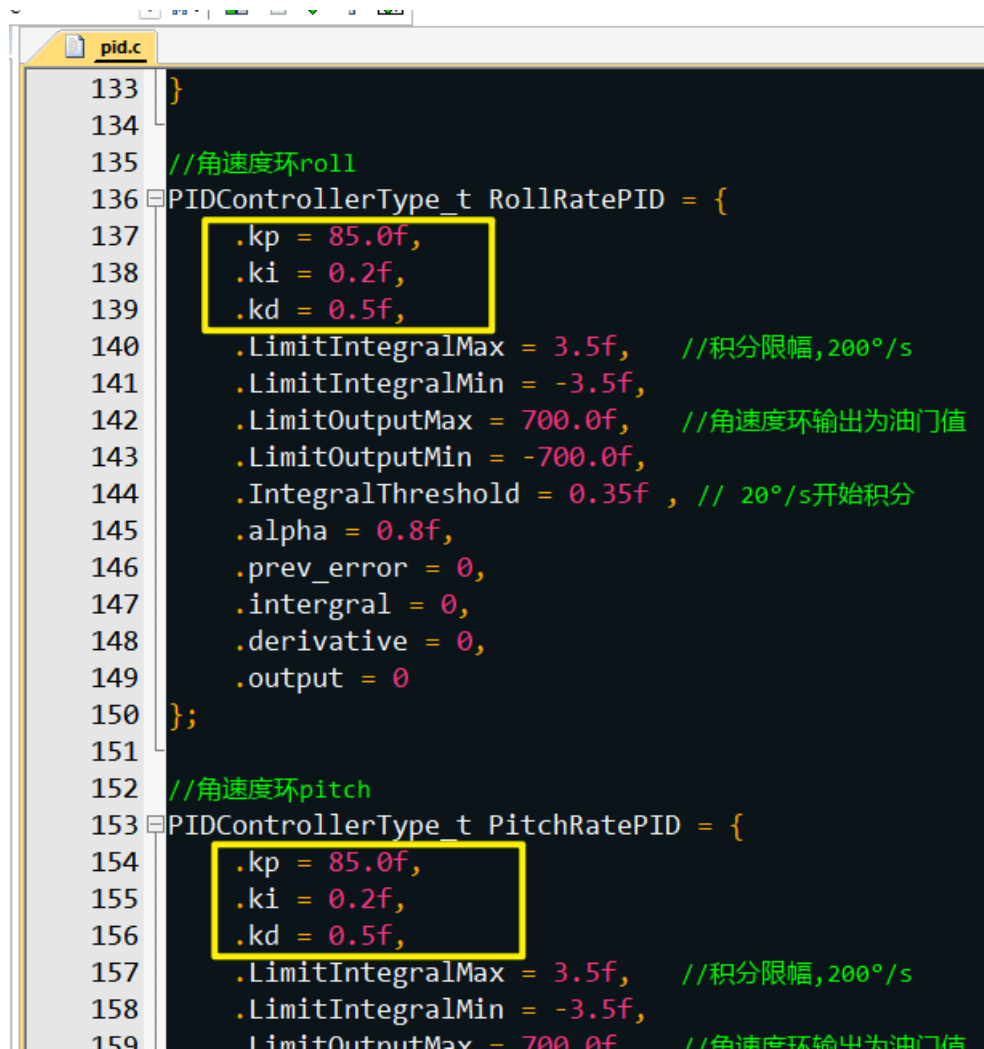
此文件是蓝牙APP的相关内容，可以对APP的操作进行解析。

- balance_task.c

此文件是四轴飞行器的主要逻辑任务，平衡功能由此任务完成，内含平衡的算法，传感器数据的使用等。如果需要研究飞行器的稳态以及算法内容，则需要仔细阅读浏览此文件学习。

- pid.c

PID算法版本才有的文件，内含PID算法，以及PID控制的定义。如需要修改PID参数，可以在此文件内进行修改。如下图示：



```
133 }
134
135 //角速度环roll
136 PIDControllerType_t RollRatePID = {
137     .kp = 85.0f,
138     .ki = 0.2f,
139     .kd = 0.5f,
140     .LimitIntegralMax = 3.5f, //积分限幅,200°/s
141     .LimitIntegralMin = -3.5f,
142     .LimitOutputMax = 700.0f, //角速度环输出为油门值
143     .LimitOutputMin = -700.0f,
144     .IntegralThreshold = 0.35f, // 20°/s开始积分
145     .alpha = 0.8f,
146     .prev_error = 0,
147     .integral = 0,
148     .derivative = 0,
149     .output = 0
150 };
151
152 //角速度环pitch
153 PIDControllerType_t PitchRatePID = {
154     .kp = 85.0f,
155     .ki = 0.2f,
156     .kd = 0.5f,
157     .LimitIntegralMax = 3.5f, //积分限幅,200°/s
158     .LimitIntegralMin = -3.5f,
159     .LimitOutputMax = 700.0f, //角速度环输出为油门值
```